

Google CTF 2021 Crypto - Pythia

deuterium — 2021-07-21

Canonical HTML: <https://deut-erium.github.io/WriteUps/2021/google-ctf/crypto/pythia/2021-07-21-google-ctf-2021-pythia.html>

Abstract

CTF writeup: Google CTF 2021's Pythia has commitment issues: AES-GCM key multicollisions turn a partitioning oracle into a short-password recovery tool.

Challenge Description

Yet another oracle, but the queries are costly and limited so be frugal with them.
pythia.2021.ctfcompetition.com 1337

Files

[source.zip](#)

- [server.py](#)
- [dummpy_flag](#)

Server

```
max_queries = 150
query_delay = 0

passwords = [bytes(''.join(random.choice(string.ascii_lowercase) \
for _ in range(3)), 'UTF-8') for _ in range(3)]
flag = open("flag.txt", "rb").read()

def menu():
    print("What you wanna do?")
    print("1- Set key")
    print("2- Read flag")
    print("3- Decrypt text")
    print("4- Exit")
    try:
        return int(input(">>> "))
    except:
        return -1

print("Welcome!\n")

key_used = 0

for query in range(max_queries):
    option = menu()

    if option == 1:
        print("Which key you want to use [0-2]?")
```

```

try:
    i = int(input(">>> "))
except:
    i = -1
if i >= 0 and i <= 2:
    key_used = i
else:
    print("Please select a valid key.")
elif option == 2:
    print("Password?")
    passwd = bytes(input(">>> "), 'UTF-8')

    print("Checking...")
    # Prevent bruteforce attacks...
    time.sleep(query_delay)
    if passwd == (passwords[0] + passwords[1] + passwords[2]):
        print("ACCESS GRANTED: " + flag.decode('UTF-8'))
    else:
        print("ACCESS DENIED!")
elif option == 3:
    print("Send your ciphertext ")

    ct = input(">>> ")
    print("Decrypting...")
    # Prevent bruteforce attacks...
    time.sleep(query_delay)
    try:
        nonce, ciphertext = ct.split(",")
        nonce = b64decode(nonce)
        ciphertext = b64decode(ciphertext)
    except:
        print("ERROR: Ciphertext has invalid format. \
Must be of the form \"nonce,ciphertext\", where \
nonce and ciphertext are base64 strings.")
        continue

    kdf = Script(salt=b'', length=16, n=2**4, r=8, p=1, backend=default_backend())
    key = kdf.derive(passwords[key_used])
    try:
        cipher = AESGCM(key)
        plaintext = cipher.decrypt(nonce, ciphertext, associated_data=None)
    except:
        print("ERROR: Decryption failed. Key was not correct.")
        continue

    print("Decryption successful")
elif option == 4:
    print("Bye!")
    break
else:
    print("Invalid option!")
print("You have " + str(max_queries - query) + " trials left...\n")

```

Recon

Taking a look at the source, we have a few observations

- The server generates 3 passwords of 3 lower case ASCII each and uses Scrypt (a Password based key derivation function) on it to derive a 16 byte encryption key from each of the 3 byte passwords, which can be treated as deriving 3 16-byte keys from a set of $26^3 = 17576$ known randomly generated keys.
- It provides us 3 options to work with. Option 1 to set 1 of the 3 unknown randomly generated keys
- Option 3 provides as a decryption oracle, allowing us to check whether any arbitrary ciphertext of our choice decrypts successfully. (Why would it fail? more details in How decryption works)
- Option 2 is the option we dig, give the server all three passwords (hence keys) correctly, it gives back the flag.
- There are 150 queries for all 3 options, which means we have effectively 147 decryption oracles to work with, which indicates we need an average case key recovery in 49 queries.

Thinking Methodology/ Ideas to reject

1. Bruteforcing/ luck

Since the number of keys is quite small, one might be tempted to bruteforce the keys and be optimistic that he/she gets all three keys in 150 attempts.

But the fact that guessing 3 keys consecutively within 150 attempts has probability as low as $\approx 10^{-7}$ which is already out of practical server-bruteforcing further enforced by 10 s delay between each guess taking 25 mins for each bruteforce, its clearly crossed out.

2. The challenge is not about Scrypt

Again, one would be tempted to think that it could be some weakness of Scrypt or the given configuration, or some relation between the derived keys which renders GCM ez. This hypothesis can also countered easily looking at the amount of stuff going inside Scrypt :P

3. Famous attacks on AES-GCM

Again, someone sees AESGCM, they get cryptopals set 8 flashbacks.

1. Key recovery attacks on repeated nonces - Again this is possible if the server *encrypts* stuff, not check its decryption. What the attacker recovers is $E(0, K)$ not the key itself, but since key space is small, they could recover it through. Although again, this is definitely not the challenge.
2. Key recovery attacks on truncated mac - Clearly, I cant see any sort of truncation. So its out.

Possible approach

The only information we can extract from the oracle is whether the provided ciphertext forms the given tag under the key of the server. How would this help us reduce the number of queries required from 26^3 to say 50. If it was possible to get it in one shot, the challenge authors would not give a slack of 50 :P

How about there exists a (ciphertext, tag) which decrypts successfully for more than one key?

This would reduce the number of queries by half! We will just need to keep on asking ciphertexts for a 2 pair till the key of server happens to lie in that pair.

That's all, if we can get a (ciphertext, tag) which is valid for n keys, we can reduce the search to a binary search, requiring $\log_2(26^3) \approx 14.1 = 15$ queries at max making 45 queries in total. Then again why 150 and not 50?

More on it later...

How decryption works

This little detour is for the people who may be confused about Authenticated Encryption with Associated Data, and why would an arbitrary decryption of some ciphertext would fail under a given key as opposed to some other mode of AES say CBC.

So the whole idea here is that the ciphertext will comprise of three parts

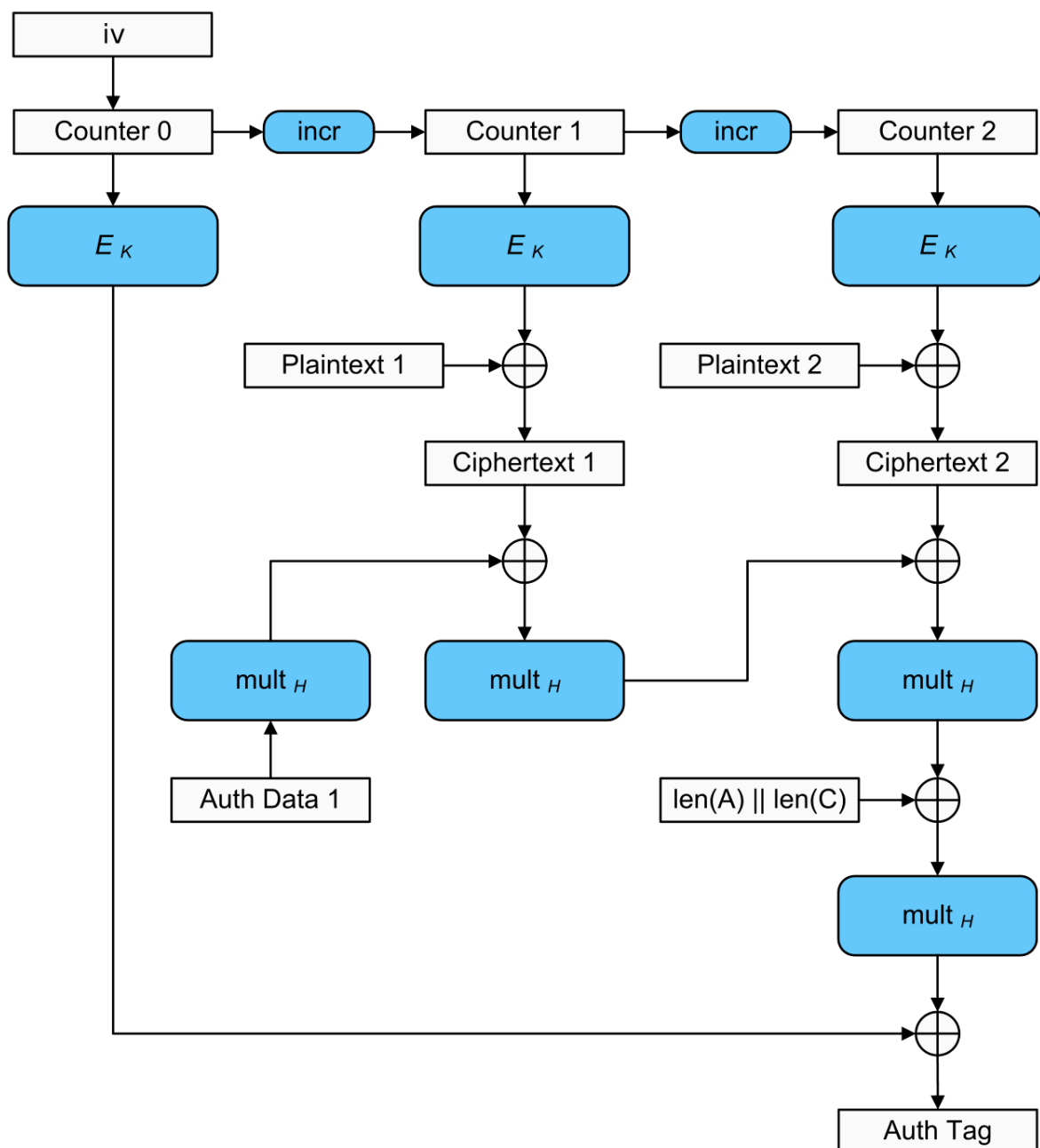
1. The encrypted payload i.e the data we wish to communicate
2. Associated un-encrypted data which contains any additional metadata which needs to be preserved against any sort of tempering.
3. Tag which is essentially an attempt of proof that the given ciphertext was encrypted by someone who holds the secret key and the payload and additional associated data has not been tempered with.

Whenever the server receives a ciphertext to decrypt, it will first try to verify that the received tag is actually corresponding to the received ciphertext encrypted with the secret key.

If the computation of tag on the ciphertext fails, the server would reject the proposal to decrypt the message.

How AES-GCM works?

AES-GCM is, put simply, an authentication mechanism built upon AES in CTR mode (a stream cipher not necessary for our discussion), such that encrypted blocks of the ciphertext, additional authentication data, and lengths of those two put together in a GCM mac. This is what's all we will need to consider. Here comes the mandatory picture from Wikipedia. Just follow the components used in the Auth tag generation.



GCM is simply a polynomial (in $GF(2^{128})$) constructed using the blocks of authentication data, ciphertext, and two additional blocks, one constructed using the lengths of data and ciphertext and one using the encryption of 96 bit nonce N appended with 31 bits of 0 and a single bit 1. i.e. $s = E(N || 0^{31}1, K)$

This polynomial is evaluated at $h = E(0, K)$ to compute the Auth tag

$$T = ((((((h * A_0) \oplus A_1) * h \dots \oplus A_m) * h \oplus C_0) * h \dots \oplus C_{n-1}) * h \oplus L) * h \oplus s$$

Since there's no additional data in the challenge, we get

$$T = (((h * C_0) \oplus C_1) * h \dots \oplus C_{n-1}) * h \oplus L) * h \oplus s$$

or

$$T = C_0 * h^{n+1} \oplus C_1 * h^{n+1} \dots \oplus C_{n-1} * h^2 \oplus L * h \oplus s$$

Attack

Continuing and exploring the idea above one would come across a recent paper titled Partitioning Oracle Attack and what's cherry on the top is that a quick CTRL+F for github in the paper reveals the POC demo of the same making it a lot easier to implement.

Construction

Continuing from the expression of tag, the terms dependent on key for calculation of tag are **h**, **s** only.

$$T = C_0 * \mathbf{h}^{n+1} \oplus C_1 * \mathbf{h}^n \dots \oplus C_{n-1} * \mathbf{h}^2 \oplus L * \mathbf{h} \oplus \mathbf{s}$$

$$C_0 * \mathbf{h}^{n+1} \oplus C_1 * \mathbf{h}^n \dots \oplus C_{n-1} * \mathbf{h}^2 = T \oplus L * \mathbf{h} \oplus \mathbf{s}$$

$$C_0 * \mathbf{h}^{n-1} \oplus C_1 * \mathbf{h}^{n-2} \dots \oplus C_{n-1} = (T \oplus L * \mathbf{h} \oplus \mathbf{s}) * \mathbf{h}^{-2}$$

writing $(T \oplus L * \mathbf{h} \oplus \mathbf{s}) * \mathbf{h}^{-2}$ as a key dependent quantity **B** we can write it for n keys $K_0 \dots K_{n-1}$, we get

$$C_0 * \mathbf{h}_0^{n-1} \oplus C_1 * \mathbf{h}_0^{n-2} \dots \oplus C_{n-1} = \mathbf{B}_0$$

$$C_0 * \mathbf{h}_1^{n-1} \oplus C_1 * \mathbf{h}_1^{n-2} \dots \oplus C_{n-1} = \mathbf{B}_1$$

$$\vdots \quad \quad \quad \vdots \quad \quad \quad \vdots \quad \quad \quad \vdots$$

$$C_0 * \mathbf{h}_{n-1}^{n-1} \oplus C_1 * \mathbf{h}_{n-1}^{n-2} \dots \oplus C_{n-1} = \mathbf{B}_{n-1}$$

$$\begin{bmatrix} 1 & \mathbf{h}_0 & \mathbf{h}_0^2 & \dots & \mathbf{h}_0^{n-1} \\ 1 & \mathbf{h}_0 & \mathbf{h}_0^2 & \dots & \mathbf{h}_0^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \mathbf{h}_0 & \mathbf{h}_0^2 & \dots & \mathbf{h}_0^{n-1} \end{bmatrix} \cdot \begin{bmatrix} C_{n-1} \\ C_{n-2} \\ \vdots \\ C_0 \end{bmatrix} = \begin{bmatrix} \mathbf{B}_0 \\ \mathbf{B}_1 \\ \vdots \\ \mathbf{B}_{n-1} \end{bmatrix}$$

Now that we have all the required equations set up, we can find $C_0, C_1 \dots, C_{n-1}$ through lagrange interpolation in $O(n^2)$ time and $O(n)$ space.

Performance considerations

Theoretically 15 searches would be enough to find the key, but it would require a multicollision for ~8000 keys.

What we can do is to first check for a few groups of smaller sizes, then proceeding with

binary search on a given group.

If we form groups of size k , the total number of calls should be roughly (for worst case number of calls)

$$17576/k + \lceil \log_2 k \rceil = 49 \implies k \approx 367$$

Time taken to find a multicollision for $k = 367$ keys,

```
import time
import statistics
k=367
times = []
for i in range(0,26**3,k):
    start_time = time.time()
    multi = multicollision(derived_keys[i:i+k])
    times.append(time.time()-start_time)
print(statistics.mean(times))
#4.06
```

It takes around 4 seconds on an i9 processor. For the binary search part the times for computation take quadratically shorter duration, adding to roughly 1.5s for 9 calls.

Overall it should take average case $(10 + 4) * (17576/367) * 1/2 + 9 * 10 + 1.5 \approx 426s \approx 7mins$ to find a key and worst case to around 760s ~ 12.7 minutes

Average case time to solve the challenge would be around 21 minutes and worst case to around 38 minutes :(

Borrowing logic from [collide_gcm.sage](#), [local copy](#) here is a dirty script

Solve script

```
import random
import string
import time

from base64 import b64encode, b64decode
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.scrypt import Scrypt
from Crypto.Cipher import AES
from itertools import product
from bitstring import BitArray, Bits
import pwn

derived_keys = [] # keys derived from scrypt of password
rev_keys = {} # holds mapping from derived key to password
for k in product(string.ascii_lowercase, repeat=3):
    kdf = Scrypt(salt=b'', length=16, n=16, r=8,
                 p=1, backend=default_backend())
    derived_key = kdf.derive("".join(k).encode())
    derived_keys.append(derived_key)
    rev_keys[derived_key] = "".join(k)
```

```

HOST, PORT = "pythia.2021.ctfcompetition.com", 1337
REM = pwn.remote(HOST, PORT)

def bytes_to_element(val, field, a):
    """Converting a bytes object to an element in `field`"""
    bits = BitArray(val)
    result = field.fetch_int(0)
    for i in range(len(bits)):
        if bits[i]:
            result += a**i
    return result

P.<x> = PolynomialRing(GF(2))
p = x**128 + x**7 + x**2 + x + 1
GFghash.<a> = GF(2**128, 'x', modulus=p)
R = PolynomialRing(GFghash, 'x')

def multicollision(keyset, nonce=b'\x00' * 12, tag=b'\x01' * 16):
    """main function to find multicollisions, Tag is kept a constant
    and so is nonce for all the key encryptions
    x_bf corresponds byte object x transformed to the field element
    """
    L_bytes = int(len(keyset) * 128).to_bytes(16, 'big')
    L_bf = bytes_to_element(L_bytes, GFghash, a)
    nonce_plus = nonce + bytes([0, 0, 0, 1])
    tag_bf = bytes_to_element(tag, GFghash, a)
    pairs = []
    for k in keyset:
        # compute H
        aes = AES.new(k, AES.MODE_ECB)
        H = aes.encrypt(b'\x00' * 16)
        h_bf = bytes_to_element(H, GFghash, a)

        s = aes.encrypt(nonce_plus)
        s_bf = bytes_to_element(s, GFghash, a)
        # assign (lens * H) + s + T to b
        b = (L_bf * h_bf) + s_bf + tag_bf
        # get pair (H, b*(H^-2))
        y = b * h_bf**-2
        pairs.append((h_bf, y))
    # compute Lagrange interpolation
    f = R.lagrange_polynomial(pairs)
    ct = ''
    for coeff in f.list()[::-1]:
        ct_pad = ''.join(map(str, coeff.polynomial().list()))
        ct += Bits(bin=ct_pad.ljust(128, '0'))
    ct = ct.bytes
    return ct + tag

def decrypt_text(text):
    REM.sendline(b'3')
    REM.sendline('A' * 16 + ',' + pwn.b64e(text))
    data = REM.recvuntil(b'Exit\n>>> ')
    return b'successful' in data

```

```

def search(size=367):
    start_time = time.time()
    api_count = 0
    for i in range(0, 26**3, size):
        print("trying range ({},{}).format(i, i + size))
        api_count += 1
        if decrypt_text(multicollision(derived_keys[i:i + size])):
            break
    lo, hi = i, i + size
    while lo <= hi:
        mid = (lo + hi) // 2
        api_count += 1
        print("trying range ({},{}).format(lo, hi))
        if decrypt_text(multicollision(derived_keys[lo:mid + 1])):
            hi = mid - 1
        else:
            lo = mid + 1
    if decrypt_text(multicollision(derived_keys[lo:lo + 1])):
        keyindex = lo
    else:
        keyindex = lo + 1
    password = rev_keys[derived_keys[keyindex]]
    print("key:{} found in {} calls".format(password, api_count))
    print("time taken :", time.time() - start_time)
    return password

REM.recvuntil(b'Exit\n>>>')
password = ""
for key_index in range(3):
    REM.sendline(b'1') # option1
    REM.sendline(str(key_index))
    REM.recvuntil(b'Exit\n>>>')
    password += search()

REM.sendline(b'2')
REM.sendline(password)
print(REM.recvregex(b'CTF{.*}'))
# CTF{gCm`ls`n0t`v3ry`r0bust`4nd`1`sh0uld`us3`s0m3th1ng`els3`h3r3}

```

And we get our flag!

CTF–gCm`ls`n0t`v3ry`r0bust`4nd`1`sh0uld`us3`s0m3th1ng`els3`h3r3”

Interactive controls are omitted from this PDF. See the canonical HTML version.